

Week 3 · Surge Speed Control

USV Guidance, Navigation and Control (Graduate) · Department of Autonomous Vehicle System Engineering, Chungnam National University | revised 2026-09-21

★ Reference material — read this first

The five courses below are **taught by the instructor of this course** and are the assumed background for it. They run from the fundamentals down to the graduate material, so start wherever the gap is. Every frame, symbol and derivation used here is developed in them at length; anyone whose prerequisites are thin should work through them first, then return.

#	Course	Level	Lang.	Video	Slides and code
1	Control Engineering (제어공학) — the foundation: transfer functions, feedback, stability, root locus, PID	undergraduate	KO	playlist	drive
2	Control System Design (제어시스템설계) — design rather than analysis: specifications, loop shaping, discrete implementation	undergraduate	KO	playlist	drive
3	Advanced Control Engineering (제어공학특론) — reference frames, the 6-DOF equation of motion, rotation matrices and Euler angles, linearisation and trim, vehicle control design	graduate	EN	playlist	drive
4	Sensor Signal Processing and Fusion (센서신호처리 및 융합) — sensor models, noise, estimation and multi-sensor fusion	graduate	EN	playlist	drive
5	Capstone Design (캡스톤디자인) — a vehicle project carried end to end	undergraduate	KO	playlist	drive

Not fluent in MATLAB or Simulink yet? Do these before Part 2. Every laboratory in this course is Simulink, and the Onramp courses are free and take a few hours each.

Tool	Where to start
MATLAB	MATLAB Onramp · Core MATLAB Skills
Simulink	Simulink Onramp · instructor's Simulink lectures part 1 · part 2

🔧 Getting the course files, and keeping them current (Windows)

The notes, models and scripts are kept in one Git repository that is updated through the semester. Clone it once; before every class, pull. A pull downloads only what has changed since the last one.

When	Where to run it	Command
once	PowerShell — installs Git for Windows	<code>winget install --id Git.Git -e</code>
once	the folder that will hold the course, e.g. <code>Documents</code>	<code>git clone https://github.com/wkyouncnu/Sensor-Signal-Processing-and-Fusion.git</code>
before every class	inside the cloned folder <code>Sensor-Signal-Processing-and-Fusion</code>	<code>git pull</code>

- `git pull` prints `Already up to date.` when nothing has changed, and otherwise lists the files it updated.
- The repository is private. When Git asks, sign in with the GitHub account the instructor has given access.
- Experiment on copies, not on the cloned files: copy a week's `WXX_simulink` folder elsewhere first, and a pull can never collide with local edits. If it already has, `git stash`, then `git pull`, then `git stash pop` sets the edits aside, updates, and puts them back.
- The MSS toolbox is not part of the repository. The weeks that simulate the Otter need it at `Tools\MSS` inside the cloned folder.

- **Course:** USV Guidance, Navigation and Control (Graduate)
- **Department:** Autonomous Vehicle System Engineering, Chungnam National University
- **This week:** ① the Week 2 controller around the Otter's surge speed, with no model of the vessel ② P, I and D read off the Scope, one at a time ③ an unreachable speed and anti-windup ④ the tuning order of Week 2, followed step by step on the vessel

★ Prerequisites from the previous week

- From Week 2: the five time-domain metrics (§2-4), what P, I and D each do (§2-6 to §2-8), the filtered derivative, back-calculation anti-windup (§2-11) and the tuning order (§2-12). This week uses all of them unchanged.
- From Week 1: the Otter model and the propeller curve $T = kn|n|$: a shaft speed produces a thrust.
- Nothing else. This week deliberately tunes **without a model of the plant**: every gain is chosen from what the Scope shows.

Learning Outcomes

Upon completion of this week, the learner is able to:

1. Characterise a plant from the outside — its steady gain, its speed and its limits — by applying a step input and reading the Scope.
2. Predict, before running it, how P, I and D will each change the speed response, using what they did in Week 2 and one difference in the plant.
3. Explain why the derivative term makes a speed loop worse, and decide from a measurement to leave it out.
4. Reproduce integrator windup on the vessel with an unreachable speed command, and remove it with back-calculation.

5. Tune the speed loop by the Week 2 tuning order, stating the measurement that fixed each gain, and check the thrust against the limit.

Prerequisites and Setup

Item	Requirement
MATLAB	R2024b, with Simulink
MSS	<code>Tools/MSS</code> , found by <code>W03_0_setup</code> through <code>mss_path</code>
Course folder	<code>lectures/W03_simulink</code>
Models	one per example — <code>W03_C_open_loop</code> , <code>W03_D_P</code> , <code>W03_E_PID</code> , <code>W03_F_windup</code> , <code>W03_G_tuning</code> — all generated by <code>W03_1_build_speed</code> , never edited by hand
Time	lecture 40 min, laboratory 1 h 20 min

Part 1 · Theory

3-1. The same controller, a new plant

This section answers: what changes when the controller of Week 2 is placed around a vessel, and what does not?

Nothing in the controller changes. Every model of this week is a Week 2 model with the mass-spring-damper removed and the vessel put in its place. The error, the three terms, the derivative filter, the limit and the back-calculation path are the same blocks in the same places.

The plant becomes a chain of blocks. The controller computes a surge force $\mathbf{X}$; the vessel has two propellers, each turned at a shaft speed $\mathbf{n}$. The chain between them is:

Block in the model	What it does	Where it comes from
thrust limit	holds $\mathbf{X}$ inside $[-133.42, 239.36]$ N	the largest thrust the two propellers produce forward and backward; derived in Appendix A1
half each	gives half of $\mathbf{X}$ to each propeller	going straight, both propellers do the same work
shaft speed	turns a thrust $\mathbf{T}$ into the shaft speed that produces it, $\mathbf{n} = \text{sign}(\mathbf{T})\sqrt{ \mathbf{T} /k}$	the propeller curve $\mathbf{T} = k\mathbf{n} \mathbf{n} $ of Week 1, solved for $\mathbf{n}$; k differs forward and backward
Otter	the MSS model <code>otter.m</code> , twelve states	Week 1
surge speed $\mathbf{u}$	picks the first state, the surge speed $\mathbf{u}$	Week 1: $\mathbf{x} = [\mathbf{u} \ \mathbf{v} \ \mathbf{w} \ \mathbf{p} \ \mathbf{q} \ \mathbf{r} \ \dots]$

The controller is tuned without a model of this chain. Week 1 could supply one, but a controller that works only when its plant is known exactly is fragile. This week follows the practitioner's route: apply a step, read the Scope, and choose each gain from what is measured — **model-free tuning**. The five metrics of Week 2 §2-4 are the language, and the tuning order of Week 2 §2-12 is the procedure.

3-2. What the vessel does, measured from outside

This section answers: what can be learned about the plant from one step input, with no equations?

Model `w03_c_open_loop` has no controller. At $t = 5$ s both propellers are asked for a constant total force $\mathbf{X}$, and the Scope shows the speed. Measured in section C:

$\mathbf{X}$ [N]	final $\mathbf{u}$ [m/s]	$\mathbf{u}$ per newton [(m/s)/N]	time to 63 % of the final speed [s]
50	0.645	0.01289	1.12
100	1.289	0.01289	1.12
200	2.579	0.01289	1.12

Three facts follow, and together they are all the tuning needs:

- The speed is proportional to the force: 0.0129 m/s per newton**, whatever the force. Holding **1.5 m/s** therefore takes about $1.5/0.0129 = 116$ N — the drag at that speed. A controller must keep supplying this force for as long as the speed is held, which is the job of the integral (§3-3).
- The response is first order.** It rises without overshoot and reaches 63 % of its final value in **1.12 s**, the **time constant**. In the language of Week 2 §2-3, the plant stores energy in one place only — the moving mass — because nothing pulls the vessel back to a position the way a spring does.

- **The speed has a ceiling.** The largest force, **239.36** N, can hold at most $239.36 \times 0.01289 = 3.09$ m/s. No controller can command more; §3-4 asks for more on purpose.

3-3. P, I and D on a speed loop

This section answers: what does each term do here, and why does one of them behave differently from Week 2?

The only difference from Week 2 is the missing spring. Everything below follows from it.

Term	On the mass-spring-damper (Week 2)	On the vessel's speed (this week)	Measured
P	a spring: faster, rings more, leaves an error	faster, leaves an error, does not ring — a first-order plant has nothing to oscillate with	section D: error 0.912 → 0.133 m/s as K_p rises 50 → 800 , overshoot at most 0.2 %
I	a patient hand that finds the spring force	a patient hand that finds the drag force	section E: every integral stops at 116.3 N, the drag at 1.5 m/s
D	a shock absorber on the position	acts on the acceleration : it pushes back whenever the vessel speeds up, as if the vessel were heavier	section E: overshoot 0.90 → 8.28 % and settling 1.30 → 4.86 s as K_d rises 0 → 100

- **Why P leaves an error.** At a steady speed the vessel needs a steady force against the drag. A proportional controller produces force only from error, $X = K_p e$, so some error must remain: at $K_p = 200$ the vessel settles at **1.081** m/s, where $200 \times 0.419 = 84$ N balances the drag at that lower speed ($1.081/0.0129 = 84$ N).
- **Why D hurts.** On a position loop the derivative of the error is a velocity, and resisting velocity is damping. On a speed loop the derivative of the error is an acceleration, and resisting acceleration is inertia: the loop becomes more sluggish, and the integral, still charging while the vessel is slow, overshoots. The same term, on a different axis, does a different job — Week 4 returns to a position-like axis, the heading, and the derivative damps again.
- **Nothing here required a model.** Each row is predicted from Week 2 and one observation of §3-2, and confirmed on the Scope.

3-4. The thrusters have a limit: windup on the vessel

This section answers: what happens when the command asks for more than the thrusters can give?

Model **w03_F_windup** commands **3.5** m/s at $t = 5$ s — faster than the **3.09** m/s ceiling of §3-2 — and a reachable **1.5** m/s at $t = 30$ s. From 5 to 30 s the thrusters sit on their limit and the error never closes. This is the experiment of Week 2 §2-11, on the vessel.

Measured in section F, $K_p = 200$, $K_i = 200$:

K_b	u at 30 s [m/s]	integral at 30 s [N]	back within 2 % of 1.5 m/s after [s]	lowest u afterwards [m/s]
0 (no anti-windup)	3.086	2747.2	11.20	1.500
0.2	3.086	570.0	4.34	1.500
1	3.073	238.3	1.38	1.491
5	3.070	172.3	3.10	1.405

- Every run reaches the same top speed: the thrust is the same, so the vessel is the same. What differs is what the integral **remembers**.

- Without anti-windup the integral stores **2747 N** — more than eleven times what the thrusters can give. When the command drops, full forward thrust continues until that store is spent: the vessel holds its top speed for about seven more seconds before it slows at all. On the water this is a vessel ignoring an order to slow down.
- Back-calculation drains the integral while the thrust is limited: the integral receives $K_i e + K_b(X - u)$, where $X - u$ is the force the limit cut off from the demand $u = P + I + D$ (the symbol u here is the controller's demand, the name of the sum block in the model). $K_b = 1$ returns in **1.38 s**. Too large a K_b drains it too hard: at $K_b = 5$ the integral is pulled below what cruising needs, and the vessel undershoots to **1.405 m/s**.

3-5. The tuning order, applied to the vessel

This section answers: how are the three gains chosen for this plant, without a model?

The procedure is Week 2 §2-12, step by step. The requirement: inside 2 % of **1.5 m/s** within 3 s of the step, overshoot below 5 %, thrust inside **[−133.42, 239.36] N**.

Step	What is done	What was measured	Decision
0	write down the requirement; check the plant is well behaved	§3-2: stable, first order, no delay	proceed
1	P only; choose the scale of K_p from the units	the plant needs about 116 N at 1.5 m/s, so K_p of order 100 N per m/s; above $K_p = 239.36/1.5 = 160$ the step already asks for the full thrust	start at $K_p = 200$
2	read the P response	overshoot 0.0 % , rise 0.70 s , error left 0.419 m/s ; raising K_p to 400 or 800 barely speeds the rise (0.54, 0.52 s) because the thrust is already on its limit	keep $K_p = 200$
3	add D only if the P response rings	it does not ring; section E shows D only adds overshoot	$K_d = 0$
4	add I until inside the band in time	$K_i = 50$: 13.34 s; 100 : 5.70 s; 200 : 1.40 s with 0.6 % overshoot	$K_i = 200$
5	look at the thrust	at the step the thrust sits on its limit for 0.20 s ; with anti-windup ($K_b = 1$) this is harmless. A smoothed command, $1/(s + 1)$, keeps it at 119.4 N at the cost of settling in 4.24 s	keep the step command, $K_b = 1$

The result, $K_p = 200$, $K_i = 200$, $K_d = 0$, $K_b = 1$: overshoot **0.56 %**, inside 2 % after **1.40 s**. These are the defaults in `w03_0_setup.m`. No transfer function of the vessel was written down to find them.

! Model-free does not mean theory-free

The tuning used two things learned from theory: what each term does (Week 2), and the time-domain metrics that say when to stop (Week 2 §2-4). What it did not use is a model of this particular plant. That is the situation of most controllers tuned in the field — and the reason the tuning order exists.

Part 2 · Laboratory

Every example has **its own model**, and every model looks the same as in Week 2: the controller on the left, the vessel in the middle, **one Scope** on the right with the speed on top and the force below. Open a model, press **Run**, and the Scope shows the response. Change a gain in the Command Window and press Run again.

Section	Model	What it shows
C	W03_C_open_loop	a step force, no controller
D	W03_D_P	P only
E	W03_E_PID	P + I, then D
F	W03_F_windup	an unreachable speed, then a reachable one
G	W03_G_tuning	the tuned loop, with a switch for a smoothed command

Each section ends with an **In class** note: purpose, what to point at, a question with its answer, and the sentence to take away.

A. Setting up (10 min)

```
cd lectures/W03_simulink
W03_0_setup
W03_1_build_speed
```

Expected output:

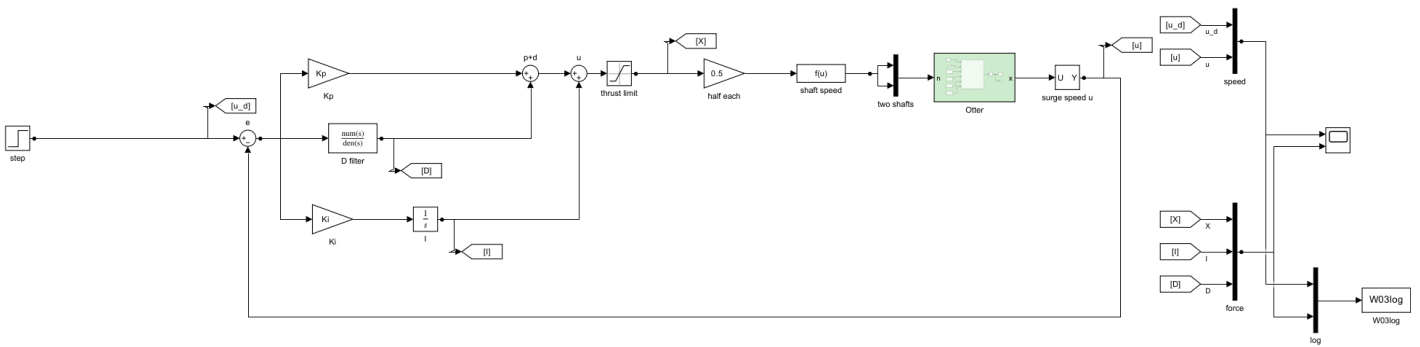
```
W03_0_setup
  thrust   X in [-133.42, 239.36] N
  gains    Kp = 200   Ki = 200   Kd = 0   Nf = 20   Kb = 1
  command  u_d = 0 -> 1.5 m/s at t = 5 s

built W03_C_open_loop.slx (overlapping lines: 0)
built W03_D_P.slx         (overlapping lines: 0)
built W03_E_PID.slx       (overlapping lines: 0)
built W03_F_windup.slx    (overlapping lines: 0)
built W03_G_tuning.slx    (overlapping lines: 0)
```

- `W03_0_setup.m` is the only file edited by hand. It also finds MSS; if it stops with an error from `mss_path`, the MSS toolbox is not at `Tools\MSS`.
- The first line of `W03_0_setup` is `clear`. Run it again whenever a Scope does not match these notes.

B. The models (10 min)

```
open_system('W03_E_PID')
```



WEEK 3, SECTION E - P+I+D
 $e = u_d - u$, $X = K_p e + K_i \int e dt + K_d \frac{de}{dt}$
 X is limited to $[X_{lo}, X_{hi}]$ by the thrusters (about -133 N to 239 N)
 Change a gain in the Command Window (e.g. $K_p = 400$) and press Run; the Scope shows the speed (top) and the force (bottom) at once.

In the figure	Meaning
<code>step</code> → <code>e</code>	the speed command u_d and the error $e = u_d - u$
<code>Kp</code> , <code>D filter</code> , <code>Ki</code> → <code>I</code> , <code>p+d</code> , <code>u</code>	the PID of Week 2, block for block; <code>u</code> is the sum of the three terms, the force demanded
<code>thrust limit</code>	the force the propellers can give, $[-133.42, 239.36]$ N
<code>half each</code> → <code>shaft speed</code> → <code>two shafts</code>	the force split between the two propellers and turned into shaft speeds
green <code>Otter</code>	the MSS vessel model
<code>surge speed u</code> and the long line along the bottom	the speed, fed back to <code>e</code>
<code>speed</code> , <code>force</code> → Scope	top: u_d and u ; bottom: the force X , the integral I , the derivative D

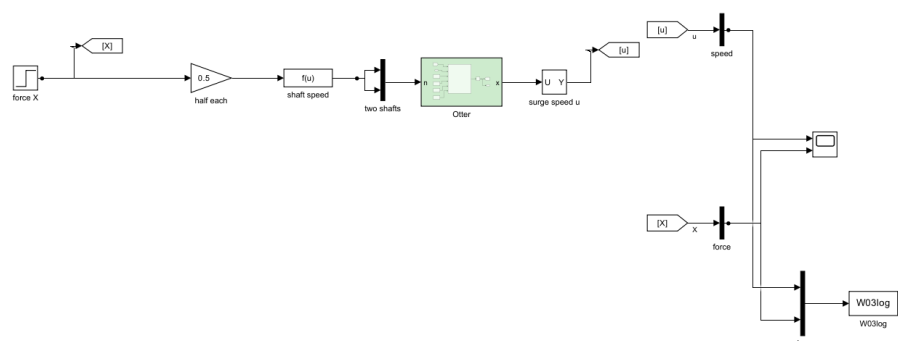
In class

- **Purpose** — show that the controller is literally the Week 2 controller; only the green block and the blocks in front of it are new.
- **Point to** — the `thrust limit`, present in every model: a real actuator is never unlimited, so the models never pretend it is.
- **Ask** — "Why is the force halved and then square-rooted?" Two propellers share the force, and each propeller's thrust grows with the square of its shaft speed.
- **Take away** — a controller is not tied to its plant; the same blocks move from a spring to a vessel.

C. The plant seen from outside (10 min)

```
open_system('W03_C_open_loop')           % press Run; then X_open = 200, Run
W03_C_plant_from_outside
```

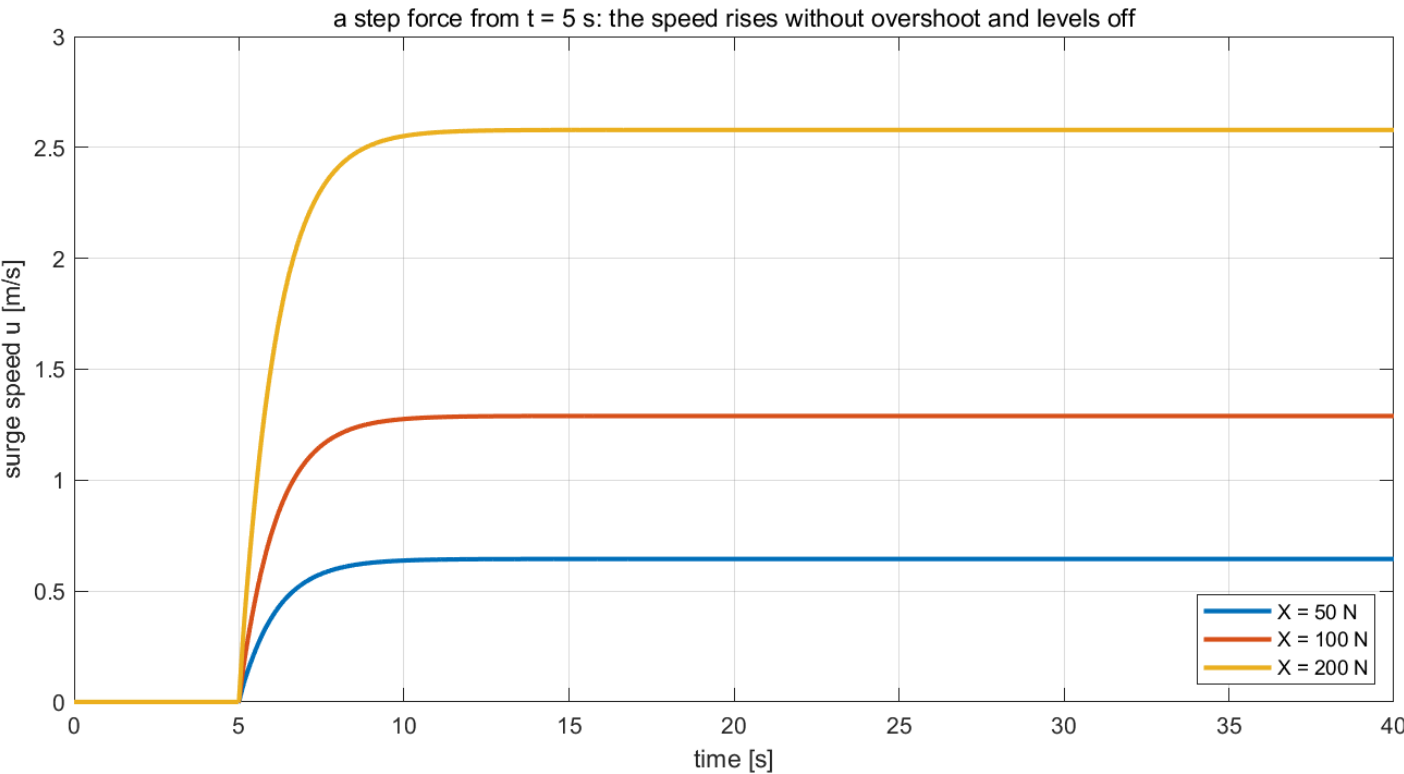
WEEK 3, SECTION C - THE PLANT, SEEN FROM OUTSIDE (no controller)
 X_{open} newtons from both thrusters at $t = t_{step}$; the Scope shows the surge speed u .
 Change X_{open} (50, 100, 200) and press Run; how fast, and how far, does u go?



In the figure	Meaning
force X	a step force of X_{open} newtons at $t = 5$ s
half each → shaft speed → Otter → surge speed u	the plant chain of §3-1
Scope	top: the speed u ; bottom: the force X

Expected output:

W03 C the plant seen from outside (no controller)			
X [N]	final u [m/s]	u per newton [(m/s)/N]	time to 63 % [s]
50	0.645	0.01289	1.12
100	1.289	0.01289	1.12
200	2.579	0.01289	1.12



In the figure	Meaning
three curves	the speed after a step force of 50, 100 and 200 N at $t = 5$ s

What the figure says

- Doubling the force doubles the speed; the shape never changes. A first-order plant with a gain of **0.0129** (m/s)/N and a time constant of **1.12** s.

In class

- **Purpose** — learn the plant the way a field engineer does: push it and watch.
- **Point to** — no overshoot in any curve; the 63 % point at the same **1.12** s for all three forces.
- **Ask** — "How much force holds 1.5 m/s?" About $1.5/0.0129 = 116$ N. That number returns in section E as the value the integral settles at.
- **Ask** — "What is the fastest speed any controller can reach?" $239.36 \times 0.0129 = 3.09$ m/s.
- **Take away** — one step input gives the gain, the speed and the ceiling; that is enough to tune.

D. P only (10 min)

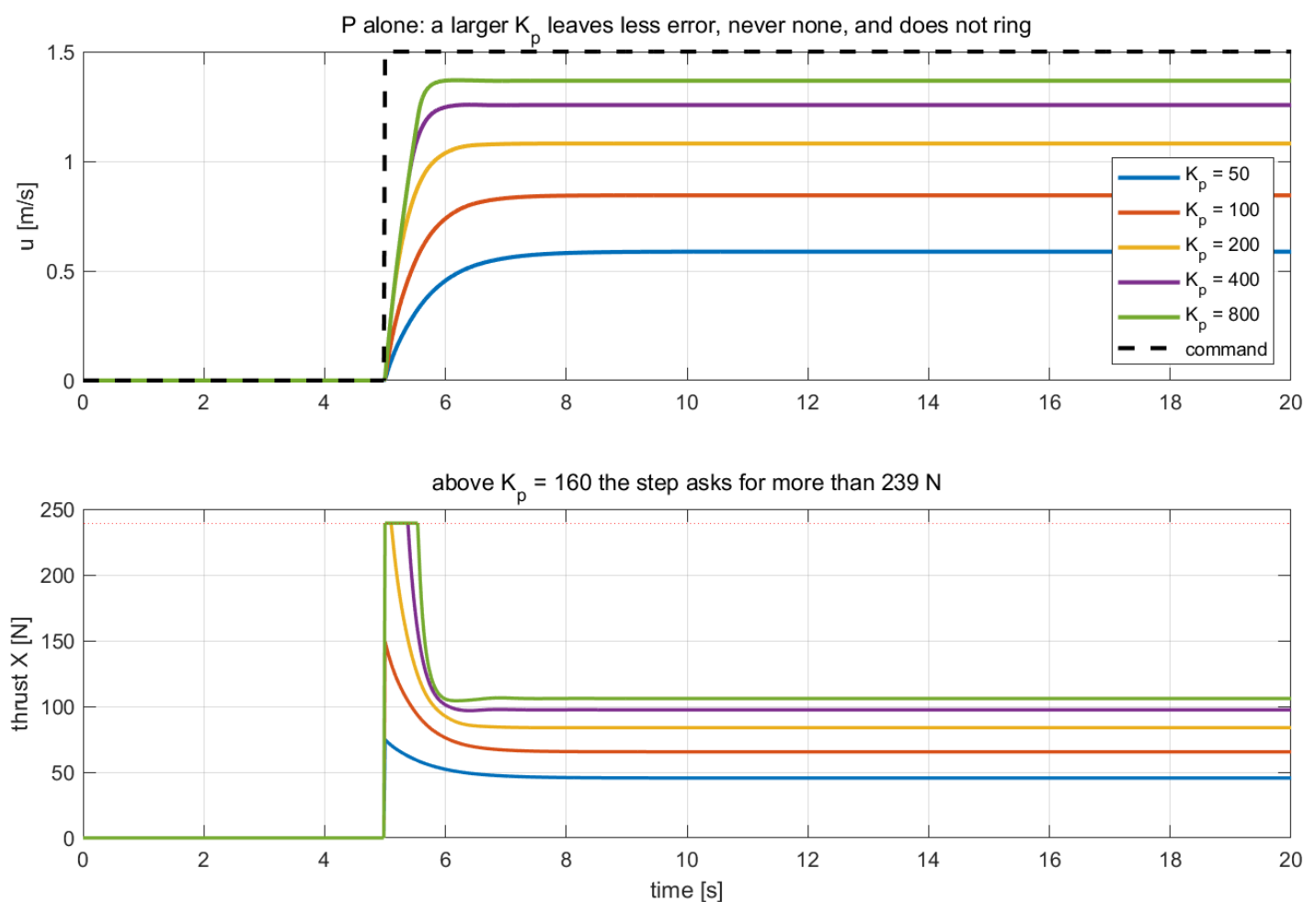
```
open_system('W03_D_P')
W03_D_proportional_only
```

% press Run; then $K_p = 800$, Run

Expected output:

W03 D P only ($u_d = 1.5$ m/s at $t = 5$ s)

K_p	final u	error left	overshoot %	rise [s]	settle [s]	time at thrust limit [s]
50	0.588	0.912	0.0	1.46	2.60	0.00
100	0.845	0.655	0.0	1.06	1.86	0.00
200	1.081	0.419	0.0	0.70	1.18	0.12
400	1.256	0.244	0.1	0.54	0.84	0.38
800	1.367	0.133	0.2	0.52	0.72	0.56



In the figure	Meaning
top	the speed for $K_p = 50$ to 800 and the dashed command
bottom	the force; the dotted red line is the 239.36 N limit

What the figure says

- A larger gain leaves less error and never none, and no gain makes the speed ring. From $K_p = 200$ on, the step drives the thrust to its limit, and more gain buys less and less speed.

In class

- **Purpose** — see P on a plant without a spring: the error of Week 2 remains, the ringing does not.
- **Point to** — the error column falling but never reaching zero; the rise-time column flattening (**0.70, 0.54, 0.52** s) as the thrust reaches its limit.
- **Ask** — "Why no overshoot even at $K_p = 800$?" The vessel stores energy only as motion; there is no spring to throw it back.
- **Ask** — "Why does raising K_p from 400 to 800 hardly help the rise time?" The thrusters are already giving all they have; the gain can ask for more, the vessel cannot deliver it.
- **Take away** — P alone always leaves a speed error, and gains beyond the actuator's reach are wasted.

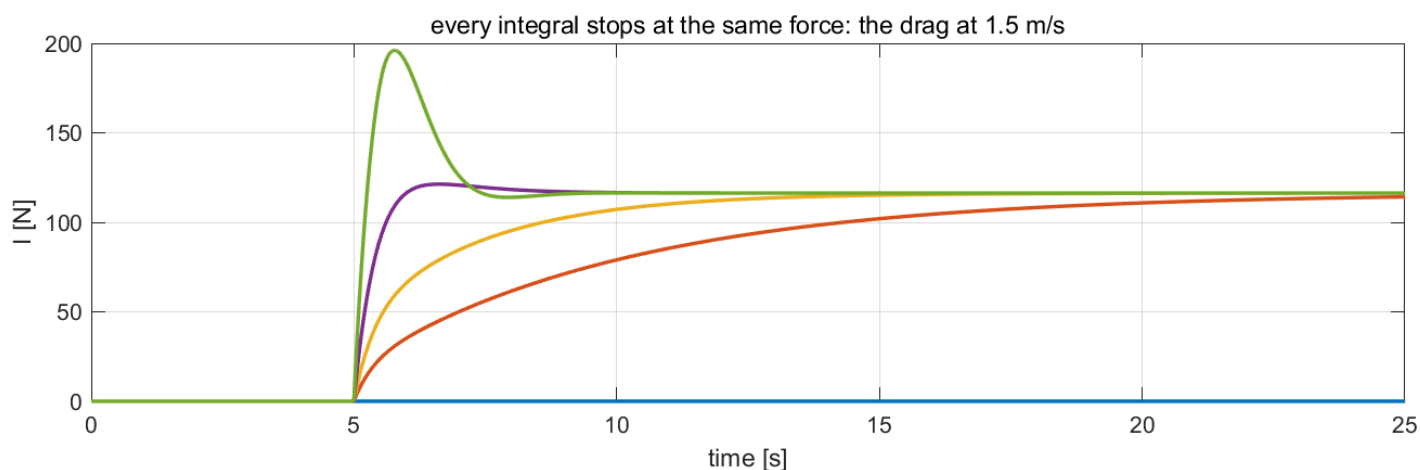
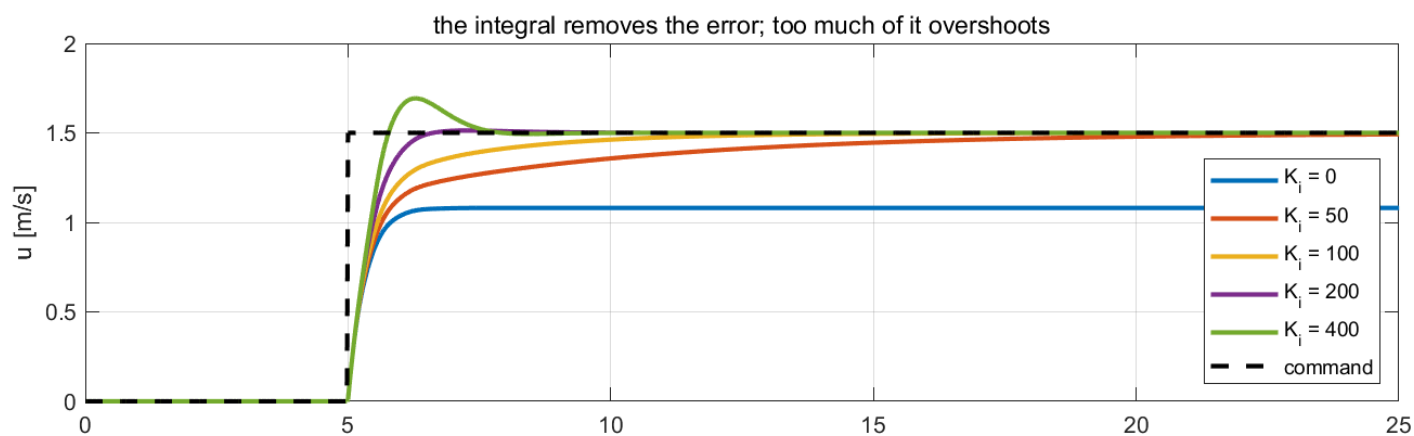
E. Add I, then try D (15 min)

```
open_system('W03_E_PID') % press Run; then Ki = 50, Run; then Ki = 200, Kd = 50, Run
W03_E_integral_and_derivative
```

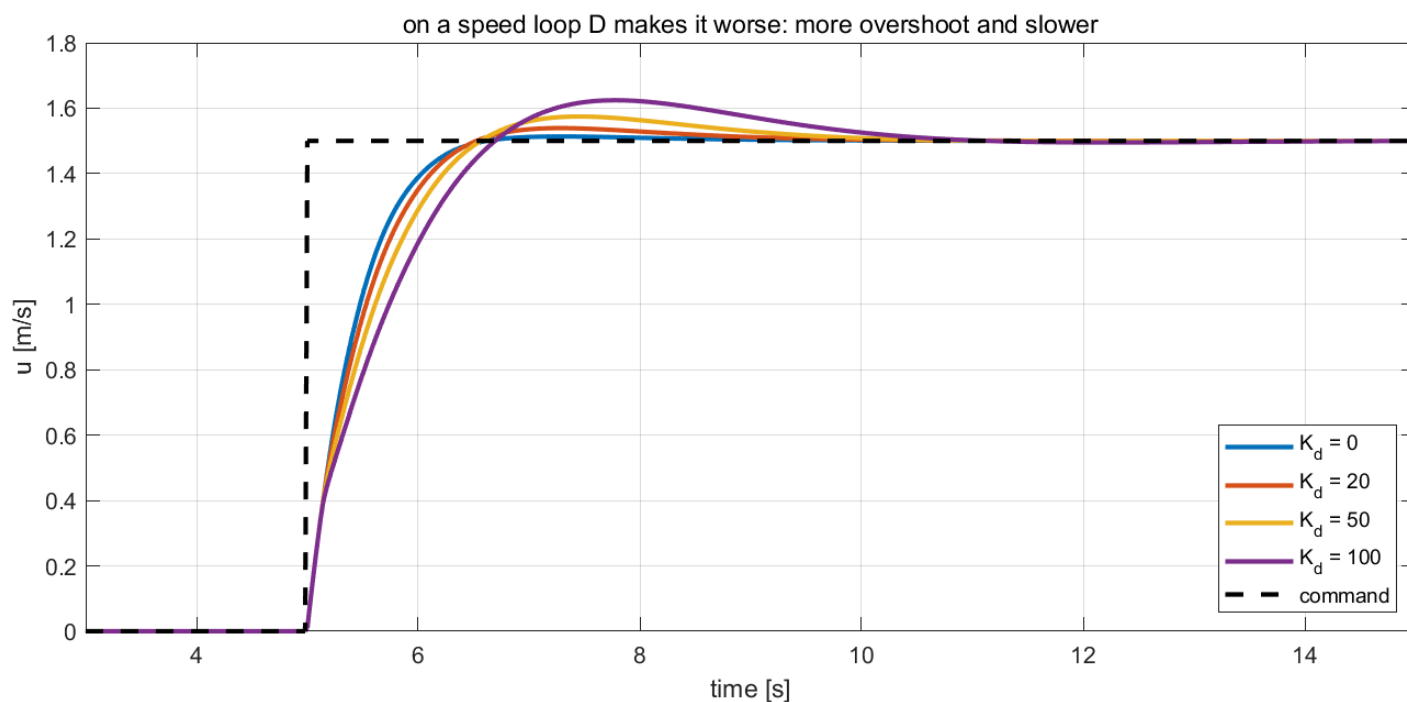
Expected output:

```
W03 E 1) the integral (Kp = 200, Kd = 0)
  Ki    u at 40 s    overshoot %    settle [s]    I at 40 s [N]
  0      1.0809         0.0         Inf         0.0
  50     1.4995         0.0        13.14        116.2
 100     1.5000         0.0         5.54        116.3
 200     1.5000         0.9         1.30        116.3
 400     1.5000        12.8         2.42        116.3

2) the derivative (Kp = 200, Ki = 200)
  Kd    overshoot %    rise [s]    settle [s]
  0         0.90         0.86         1.30
 20         2.60         0.94         2.92
 50         4.97         1.06         3.96
100         8.28         1.24         4.86
```



In the figure	Meaning
top	the speed for $K_i = 0$ to 400 at $K_p = 200$
bottom	the integral term; every curve with $K_i > 0$ levels off at the same force



In the figure	Meaning
traces	the speed for $K_d = 0, 20, 50, 100$ at $K_p = K_i = 200$

What the figure says

- The integral removes the error at any positive gain, and stops at **116.3 N** — the drag at 1.5 m/s, found without being told. Too much of it ($K_i = 400$) overshoots by **12.8 %**.
- Every derivative gain makes the response worse: more overshoot and a longer settling time.

In class

- Purpose** — see the one term that is different on this axis, and let the measurement decide.
- Point to** — the integral levelling off at 116 N, the number predicted in section C from the open loop; the derivative curves rising later and overshooting more as K_d grows.
- Ask** — "The derivative braked the mass in Week 2. Why does it hurt here?" The error of a speed loop changes with the acceleration; resisting acceleration is like adding mass, and a heavier vessel is slower and lets the integral overshoot.
- Ask** — "Which K_i meets 'inside 2 % within 3 s'?" $K_i = 200$: 1.30 s. At 100 it takes 5.54 s; at 400 it overshoots.
- Take away** — PI is the speed controller; D is left out because the Scope says so.

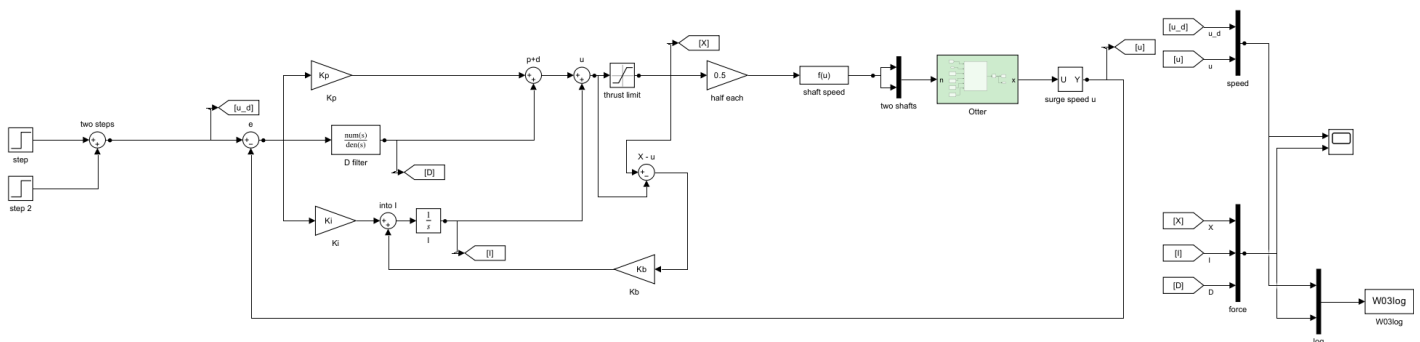
F. An unreachable speed, and anti-windup (15 min)

```
open_system('W03_F_windup')           % u_step = 3.5; t_step2 = 30; u_step2 = 1.5; T_final = 60; Run
n; then Kb = 0, Run
W03_F_unreachable_speed
```

Expected output:

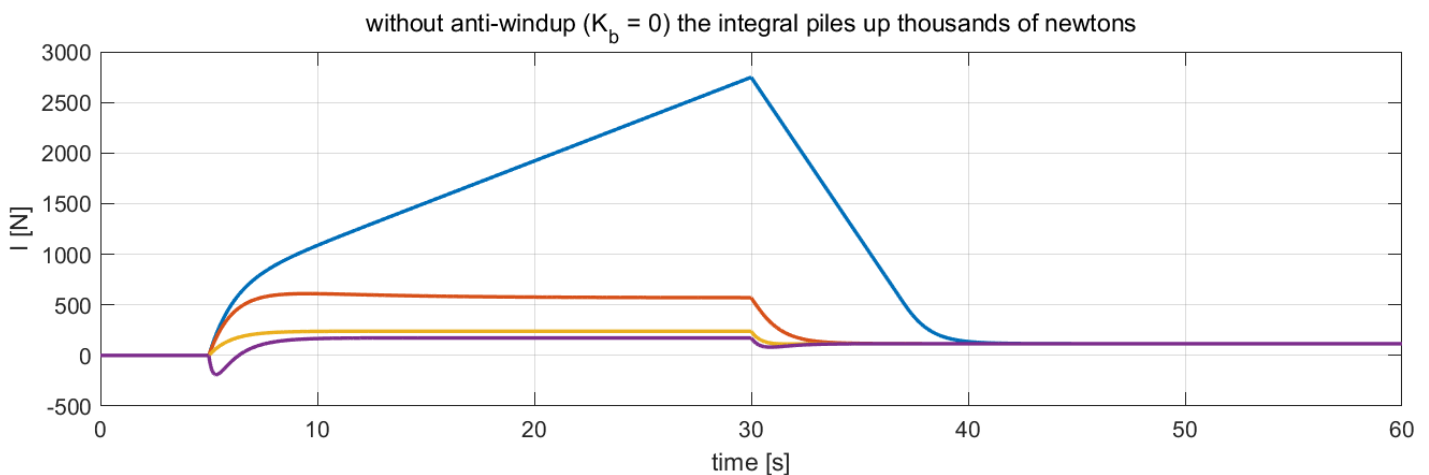
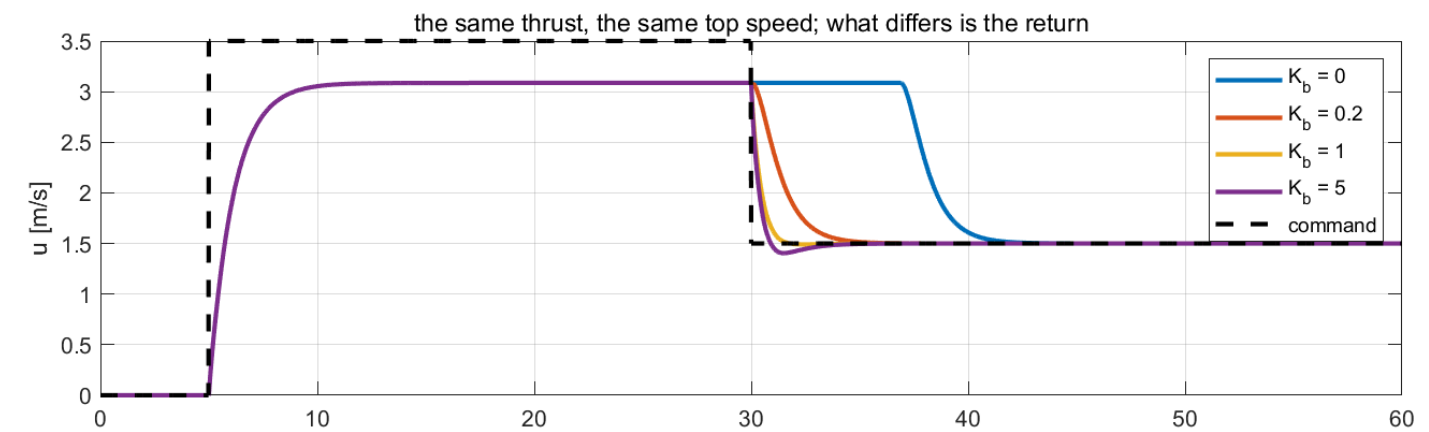
W03 F 3.5 m/s from 5 s (unreachable), 1.5 m/s from 30 s

Kb	u at 30 s	I at 30 s [N]	back within 2 % of 1.5 m/s after [s]	lowest u [m/s]
0	3.086	2747.2	11.20	1.500
0.2	3.086	570.0	4.34	1.500
1	3.073	238.3	1.38	1.491
5	3.070	172.3	3.10	1.405



WEEK 3, SECTION F - AN UNREACHABLE SPEED, AND ANTI-WINDUP
 $e = u_d - u$, $X = K_p e + K_i \int e$ (the $\int$ is $(s + M)$ e
 X is limited to $[X_{lo}, X_{hi}]$ by the Thrusters (about -133 N to 239 N)
 anti-windup: $\dot{I}_{dot} = K_i e + K_b (X - u)$, $K_b = 0$ switches it off
 $u_{step} = 3.5$ is faster than the 'Cater' can go; at t_{step2} the command drops to u_{step2}
 Change a gain in the Command Window (e.g. $K_p = 400$) and press Run;
 the Scope shows the speed (top) and the force (bottom) at once.

In the figure	Meaning
step, step 2, two steps	the command: u_{step} at 5 s, then u_{step2} at t_{step2}
$X - u$, K_b , the line back to into I	back-calculation: the force cut off by the limit, fed back to drain the integral



In the figure	Meaning
top	the speed for four values of K_b and the dashed command
bottom	the integral term

What the figure says

- Until 30 s the four runs are one. After it, the vessel without anti-windup keeps full thrust for about seven more seconds, spending a store of **2747 N**; with $K_b = 1$ it is back in **1.38 s**.

In class

- Purpose** — windup on the vessel, where it has a consequence: an order to slow down that is ignored.
- Point to** — the blue integral climbing steadily from 5 to 30 s while the speed is flat; then the flat blue speed after 30 s.
- Ask** — "Why does the integral keep growing when the speed has stopped changing?" The error, **3.5 – 3.09**, never closes, and the integral adds it up for 25 s.
- Ask** — "Why is $K_b = 5$ worse than $K_b = 1$?" It drains the integral below the **116 N** needed at 1.5 m/s, and the vessel dips to **1.405 m/s**.
- Take away** — every loop with a real actuator needs anti-windup, and a moderate K_b is enough.

G. The tuning order, step by step (15 min)

```
open_system('W03_G_tuning')           % press Run; then ref_filter = 1, Run
W03_G_tuning_by_hand
```

Expected output:

W03 G the tuning order on the Otter

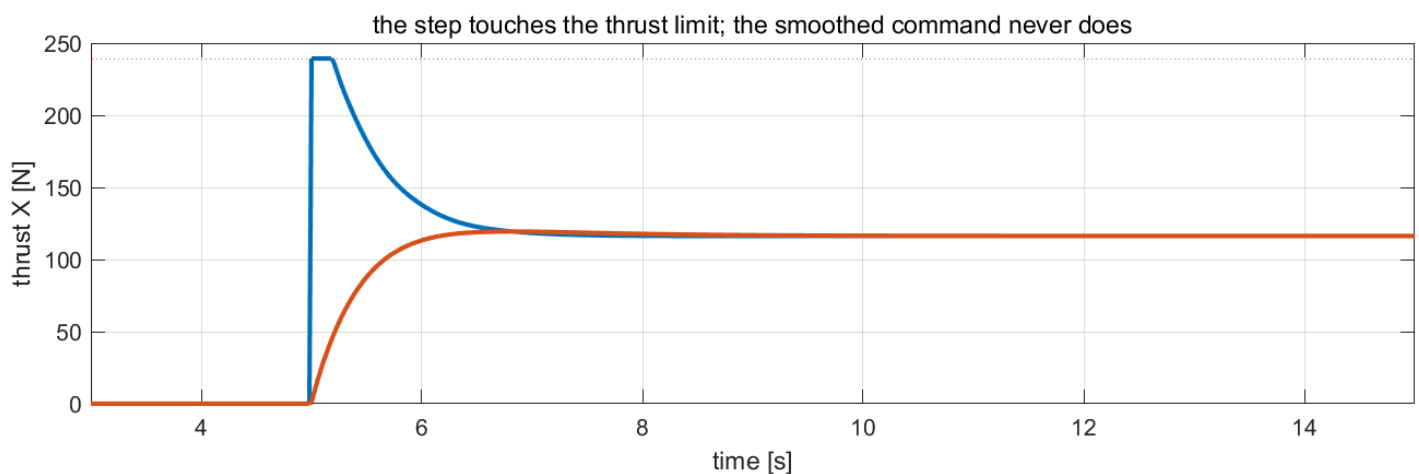
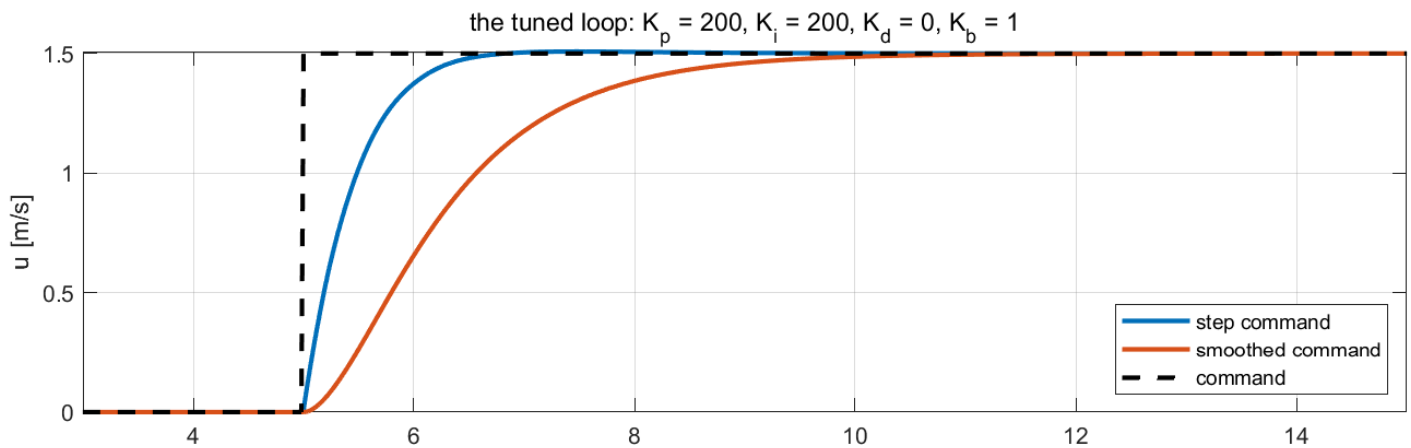
1-2 $K_p = 200$: overshoot 0.0 %, rise 0.70 s, error left 0.419 m/s

3 no ringing, so nothing for D to damp: $K_d = 0$ (section E: D only adds overshoot)

4 $K_i = 50$: 13.34 s (0.0 %) 100: 5.70 s (0.0 %) 200: 1.40 s (0.6 %) $\rightarrow K_i = 200$

5 force: 239.4 N, on the thrust limit for 0.20 s (anti-windup on); smoothed command: 119.4 N, settle 4.24 s

final: $K_p = 200$, $K_i = 200$, $K_d = 0$, $K_b = 1$: overshoot 0.56 %, inside 2 % after 1.40 s



In the figure	Meaning
top	the speed for the step command and for the command smoothed by $1/(s + 1)$
bottom	the thrust; the dotted red line is the limit

What the figure says

- The tuned loop meets the requirement with margin. The step touches the thrust limit for a fifth of a second; the smoothed command never does, and pays with a slower arrival.

In class

- **Purpose** — close the week with the full procedure on the vessel, and compare with Week 2: the same steps, different decisions, because the plant is different.
 - **Point to** — step 3, skipped here and needed in Week 2; step 4, where K_i is raised until the requirement is met and no further.
 - **Ask** — "Settling here is 1.40 s, in section E 1.30 s, with the same gains. Why?" This model has anti-windup on, and the step touches the limit; back-calculation changes the first second slightly. Both meet the requirement.
 - **Ask** — "When would the smoothed command be the right choice?" When the thrust must stay below its limit — to save power, to avoid cavitation, or because passengers feel the jolt.
 - **Take away** — requirements, P, D only if it rings, I until the band is met, then the actuator: the same order on every plant.
-

Summary

Week Summary

Step	What was done	How it was verified
1	placed the Week 2 controller around the Otter's surge speed	the models differ from Week 2 only in the plant chain; <code>check_overlaps</code> 0
2	measured the plant from outside	section C: 0.01289 (m/s)/N at every force, time to 63 % 1.12 s, ceiling 3.09 m/s
3	P alone	section D: error never zero; no ringing; more gain wasted at the thrust limit
4	I finds the drag; D hurts	section E: integral 116.3 N at every K_i ; overshoot 0.90 → 8.28 % with K_d
5	windup on the vessel	section F: 2747 N stored, 11.20 s to return without anti-windup; 1.38 s with $K_b = 1$
6	the tuning order	section G: $K_p = 200$, $K_i = 200$, $K_d = 0$, $K_b = 1$; overshoot 0.56 %, settled in 1.40 s

Progress Check

★ Minimum condition for following Week 4

Theory

- ☐ Able to state the three facts a step input gave about the vessel, and the force needed to hold 1.5 m/s.
- ☐ Able to explain why P does not ring on this plant and why D makes it worse.
- ☐ Able to explain what the integral remembers during an unreachable command, and what K_b does to it.

Laboratory

- ☐ `w03_1_build_speed` ran and every model reported `overlapping lines: 0`.
- ☐ A gain was changed in the Command Window and the model's Scope showed the new response.
- ☐ `w03_check(1)`, (2) and (3) pass on a model built from `w03_P1_start`.

Recorded observations

- ☐ The force the integral settled at in section E, and where the same number appeared in section C.
- ☐ The time to return to 1.5 m/s with and without anti-windup in section F.

In-class laboratory — close the speed loop by hand

The second hour of the Week 3 session is spent building the loop in Simulink from a model that holds only the vessel. Three problems, one hour, with a checker that compares the result against the numbers measured above.

```
cd lectures/w03_simulink/problems
w03_P1_start           % creates w03_P1.slx – hull and thrust map only
w03_check(1)          % run this whenever, as often as needed
```

	Problem	Time	The number it must reproduce
1	The open loop — a constant force in, a log out	20 min	0.6447, 1.2894, 2.5788 m/s for 50, 100, 200 N
2	Proportional control — and the error that never closes	20 min	0.8448 m/s at $K_p = 100$, against $u_d = 1.5$
3	Add the integral — the tuned gains of section G	20 min	1.5 m/s with no error left, overshoot below 5 %

- The problem sheet is [W03_simulink/problems/README.md](#) , and reference answers are in [W03_simulink/solutions/](#) .

Assignment 3

- Due:** before the Week 4 session
- Submit:** the modified [W03_0_setup.m](#) , the numbers requested below, and two figures

① Requirements

Double the payload, `mp = 50` kg, in [W03_0_setup.m](#) and [W03_vars.m](#) , and tune the speed loop again by the tuning order of §3-5 with the requirement: inside 2 % of **2.0** m/s within 3 s, overshoot below 5 %, anti-windup on.

② Verification — mandatory

★ **A claim is not a result. Numbers are required, with the tolerance band stated**

- Repeat section C with the new payload and report the speed per newton and the time to 63 %. State which of the two changed, and by how much.
- For each step of the tuning order, the measurement that decided it, as in the table of §3-5.
- The final gains, overshoot and 2 % settling time, measured with [step_metrics](#) .
- Section F with the final gains at $K_b = 0$ and $K_b = 1$: the time to return.

③ Analysis (5–10 lines)

Compare the final gains with those of §3-5. Explain, from the measurement of ②.1 alone, why the gains moved in the direction they did.

Grading

Criterion	Weight
The model runs and produces the requested output	25%
Verification performed and numbers reported	40%
Correctness of the analysis	25%
Readability of the code	10%

Troubleshooting

Symptom	Cause	Fix
<code>w03_0_setup</code> stops in <code>mss_path</code>	the MSS toolbox is not at <code>Tools\MSS</code>	place MSS there; see the first page
the speed never reaches the command, however large the gains	the command exceeds the 3.09 m/s ceiling	expected; that is the experiment of section F
the speed returns seconds late after a lower command	$K_b = 0$	set <code>Kb = 1</code>
a Scope does not match these notes	a changed variable is still in the workspace	run <code>w03_0_setup</code> again
a <code>.slx</code> was edited by hand and now differs	the models are generated	run <code>w03_1_build_speed</code> again
the build stops at the block <code>shaft speed</code> with a syntax error	a Fcn block expression used <code>sign</code> or <code>>=</code>	the Fcn block writes the sign function <code>sgn</code> and has no relational operators; the builder already does

References

Primary

- Fossen, T. I. *Handbook of Marine Craft Hydrodynamics and Motion Control*, 2nd ed., Wiley, 2021, §12.2 (PID control of marine craft) and §12.2.6 (integrator anti-windup).
- Åström, K. J. and Hägglund, T. *Advanced PID Control*, ISA, 2006, Ch. 3 (derivative filtering and anti-windup).
- MSS toolbox, `Tools/MSS/VESSELS/otter.m` — the vessel model.

In this course

- Week 2 §2-4 (the metrics), §2-6 to §2-8 (P, D and I), §2-11 (anti-windup) and §2-12 (the tuning order).
- Appendix A1 — where the thrust limit $[-133.42, 239.36]$ N comes from.
- The previous, model-based version of this week — plant identification, pole placement, the three anti-windup schemes compared with the library block — is kept in `w03_simulink/_previous_version/` for reference.

Next Week

- Week 4 — Heading Control**
- The same controller, around the heading. The heading is an angle, the integral of the yaw rate, so the plant has something like a position again, and the derivative term, harmful on the speed loop of §3-3, damps once more.
- The angle wraps at $\pm 180^\circ$, and the smallest-signed-angle function `ssa` keeps the controller from steering the long way round.